Building a Machine Learning Model to Analyse Customer Behaviour for Bank Term Deposits

Table of Contents

Building a Machine Learning Model to Analyse Customer Behaviour for Bank Term Deposits 1

Table of Contents 1

1 Introduction 2

1.1 Report Introduction 2

1.2 Dataset Identification 2

1.3 Supervised learning task identification 4

2 Exploratory Data Analysis 4

2.1 Question Identification 4

2.2 Exploratory Data Analysis Process and Results 5

2.2.6 Call Duration Analysis 9

2.3 EDA Conclusions 10

3 Experimental Design & Preprocessing 11

3.1 Algorithms 11

3.2 Evaluation Metrics 11

3.3 Preprocessing Steps 12

3.4 Limitations 17

4 Model Development 18

4.1 Data Splitting 18

4.2 Model Training 18

4.3 Initial Results 20

5 Model Tuning 23

5.1 Identifying Model Weaknesses 23

5.2 Model Improvement (Hyperparameter Tuning) 24

5.3 Improved Results 25

6 Conclusion and Reflective Commentary 27

6.1 overall Conclusion 27

6.2 Reflection on Learning and Methodology 27

6.3 Challenges, Limitations and Ethical Considerations 28

6.4 Future Work 28

6.5 Reflection on individual learning 28

7 Bibliography 29

1 Introduction

1.1 Report Introduction

To promote their financial products, the banking industry uses a variety of marketing campaigns. The banks have placed great importance on the various ways they can reach out to customers indirectly, and the collection of customer data through marketing campaigns has enabled them to do that. While banks still use some form of traditional marketing, such as sending direct mail, it can be costly and time-consuming, as the banks can send many offers to customers for whom there is no interest. Machine learning can assist banks in better predicting the likelihood that a customer would respond positively to their marketing campaigns and thus help banks save money on their overall campaigns and increase the number of new customers that they will attract to their products. The objective of this report is to analyse the potential for supervised machine learning methods to assist in predicting if a customer will take out a bank term deposit or not.

1.2 Dataset Identification

This research utilizes data collected through direct campaigns created by a bank located in Portugal from 2008-2013. The information within this data covers 4,521 records of customers along with 17 features describing their demographics (e.g. age, job, education), as well as their financial indicators (e.g. account balance, loans) and how they were contacted and how many times they were contacted during the campaign and previous campaigns to determine whether they had signed up for a term deposit (either "yes" or "no"). “Y” is the target variable indicating if a client signed up for a term deposit or not. The dataset "Bank Marketing" was given to researchers and educators by the UCI Machine Learning Repository. The data used in the Bank Marketing dataset came directly from real customers, so it is important to account for factors such as a customer's socioeconomic Class, Age group, and Communication Access, which may create biases in the Model Predictions.

Table 1: Features of the Bank Marketing Dataset

Attribute	Description
Age	A client’s age in years can be seen as representing demographic information that can impact financial decision-making.
Job	The type of job or profession (e.g., admin, technician, services, management). This indicates socioeconomic status, income stability, and career background.
Marital	The client's marital status (single, married, or divorced). It might illustrate the person's financial obligations or lifestyle that might influence the consumption of subscription services.
Education	The level of education of the client, starting from primary to university degree. Education can have an impact on financial knowledge and the willingness to invest for a long period of time.
Default	This indicates if the client has any credit in default, which can signal financial risk. It may influence the bank's decision-making process and affect how likely they are to accept offers.
Balance	The average yearly account balance of the client (in euros) gives us a glimpse into their financial stability and the resources they have available for investing in term deposits.
Housing	This indicates if the client has a housing loan, which can give insight into their financial obligations and debt levels that might influence their investment choices.
Loan	It denotes whether a customer has taken a personal loan. Like housing loans, it connects with their overall debt exposure and financial stress.
Contact	The medium of contact, or the channel employed for contact-cellular or telephone. Contact method may influence customer engagement and campaign success rates.
Day	Day of the month when the client was last contacted. May help identify temporal patterns or campaign timing effects.

Month	The month of last contact. Response rates might be affected by seasonality or campaign timing.
Duration	Duration of last contact in seconds: this is one of the most important predictors, as longer calls usually indicate genuine interest.
Campaign	Number of times called this client in the current campaign. It shows persistence of marketing efforts and potential fatigue.
P-Days	Number of days since the client was last contacted in a previous campaign (−1 indicates no prior contact). Shows past interaction history.
Previous	Number of contacts performed before this campaign. Helps measure long-term campaign engagement levels.
P-Outcome	Outcome of the previous marketing campaign (e.g., success, failure, unknown). Indicates previous customer behaviour patterns.
Y (Target)	Whether the client subscribed to a term deposit ("yes" or "no"). This is the output variable your classification model aims to predict.

2 Exploratory Data Analysis

2.1 Question Identification

The exploratory data analysis investigates by four research questions guided by an initial assumption based on behavioural economics, financial literacy and common trends in bank marketing datasets. These questions provide a foundation for effective predictive modelling.

Q1. What is the overall subscription rate, and is the target variable (y) imbalanced, how might this affect the performance of machine learning models?

Assumption: The dataset is expected to be imbalanced, with far more "no" than "yes" responses, as term deposits typically offer low returns. Research in behavioural economics shows that clients are discouraged to commit to low-return products (Meissner et al., 2022)

Q2. How do demographic characteristics (age, job, marital status, education) influence subscription behaviour?

Assumption:

- **Age:** Older clients may have higher financial stability and are likely to invest long-term and subscribe. (Silinskas, Ranta & Wilska, 2021)
- **Job:** Higher income professions may show greater subscription rates due to financial stability.
- **Education**: Higher education levels correlate with financial literacy and therefore higher subscription probability. (Zeng et al., 2025).
- **Marital Status:** Single people may have fewer financial obligations and may subscribe more frequently than married clients.

Q3. How do financial indicators (account balance, housing loan, personal loan) relate to subscription outcomes?

Assumption: Clients with higher account balances and fewer existing loans are more likely to subscribe due to better financial flexibility and lower risk. (Zeng et al., 2025).

Q4. How do campaign interactions (number of contacts, previous outcomes, call duration, days since last contact) affect a client's likelihood of subscribing?

Assumption: Longer call durations and successful prior contacts are expected to positively influence subscription rates, whereas excessive contact may reduce engagement.

2.2 Exploratory Data Analysis Process and Results

2.2.1 The dataset was initially examined using the head() function in figure 1 to display each feature and examples of the data.

```
df = pd.read_csv("bank.csv", sep=";")

df.head(10)
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	30	unemployed	married	primary	no	1787	no	no	cellular	19	oct	79	1	-1	0	unknown	no
1	33	services	married	secondary	no	4789	yes	yes	cellular	11	may	220	1	339	4	failure	no
2	35	management	single	tertiary	no	1350	yes	no	cellular	16	apr	185	1	330	1	failure	no
3	30	management	married	tertiary	no	1476	yes	yes	unknown	3	jun	199	4	-1	0	unknown	no
4	59	blue-collar	married	secondary	no	0	yes	no	unknown	5	may	226	1	-1	0	unknown	no
5	35	management	single	tertiary	no	747	no	no	cellular	23	feb	141	2	176	3	failure	no
6	36	self-employed	married	tertiary	no	307	yes	no	cellular	14	may	341	1	330	2	other	no
7	39	technician	married	secondary	no	147	yes	no	cellular	6	may	151	2	-1	0	unknown	no
8	41	entrepreneur	married	tertiary	no	221	yes	no	unknown	14	may	57	2	-1	0	unknown	no
9	43	services	married	primary	no	-88	yes	yes	cellular	17	apr	313	1	147	2	failure	no

Figure 1: Displayed features of the bank dataset

2.2.2 Target Variable Analysis

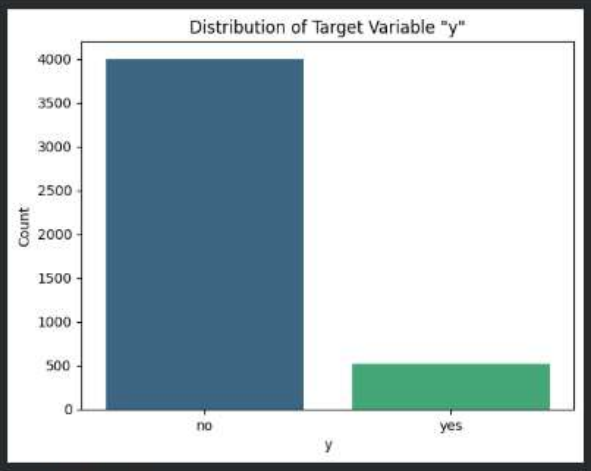
Figure 2 shows the target variable ('y') is heavily imbalanced with 88.48% of non-subscribers and 11.52% subscribing, This imbalance can skew model predictions toward the majority class, 'no', with accuracy becoming unreliable. Therefore, metrics like precision, recall, and F1-score p and AUC provide a clearer picture of model performance fairly

```
[6]
✓ 0s
sns.countplot(x='y', data=df, hue='y', palette='viridis', legend=False)
plt.title('Distribution of Target Variable "y"')
plt.xlabel('y')
plt.ylabel('Count')
plt.show()

# Calculate the distribution of the target variable 'y' in percentages
target_distribution = df['y'].value_counts(normalize=True) * 100

# Convert to a DataFrame for better display
target_distribution = target_distribution.reset_index()
target_distribution.columns = ['y', 'Percentage']

# Display the distribution
display(target_distribution)
```



	y	Percentage
0	no	88.476001
1	yes	11.523999

Figure 2: Distribution of clients' responses to the subscription target variable

2.2.3 Demographic Variables and Subscription Behaviour

Demographic variables show noticeable behavioural differences across client groups. Higher subscription rates are among older customers, clients with higher education and those who are retired or in higher paying roles, reflecting greater financial literacy and investment readiness. Tree-based models may capture these patterns a lot more effectively

2.2.4 Financial Variables

Financial characteristics show clear patterns in predicting subscriptions. Clients with higher account balances and no existing loans tend to subscribe more often, likely reflecting greater financial flexibility. Since balance values are right skewed with extreme outliers, applying transformations or using models that handle skewed data is important to prevent biased learning during training

```
[9]
✓ 1s
# Categorical Box plots

plt.figure(figsize=(18, 12))

categorical_features_for_bar = ['job', 'marital', 'education', 'housing', 'loan']

for i, feature in enumerate(categorical_features_for_bar, 1):
    plt.subplot(2, 3, i) # Adjust subplot grid as needed, e.g., 2 rows, 3 columns
    sns.countplot(data=df, x=feature, hue='y', palette='viridis')
    plt.title(f'Subscription Rate by {feature.capitalize()}')
    plt.xlabel(feature.capitalize())
    plt.ylabel('Count')
    plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
    plt.legend(title='Subscribed', labels=['No', 'Yes'])

# 6. Adjust the layout of the plots and display
plt.tight_layout()
plt.show()
```

Figure 3.1 The relationship between demographics and subscriptions levels.

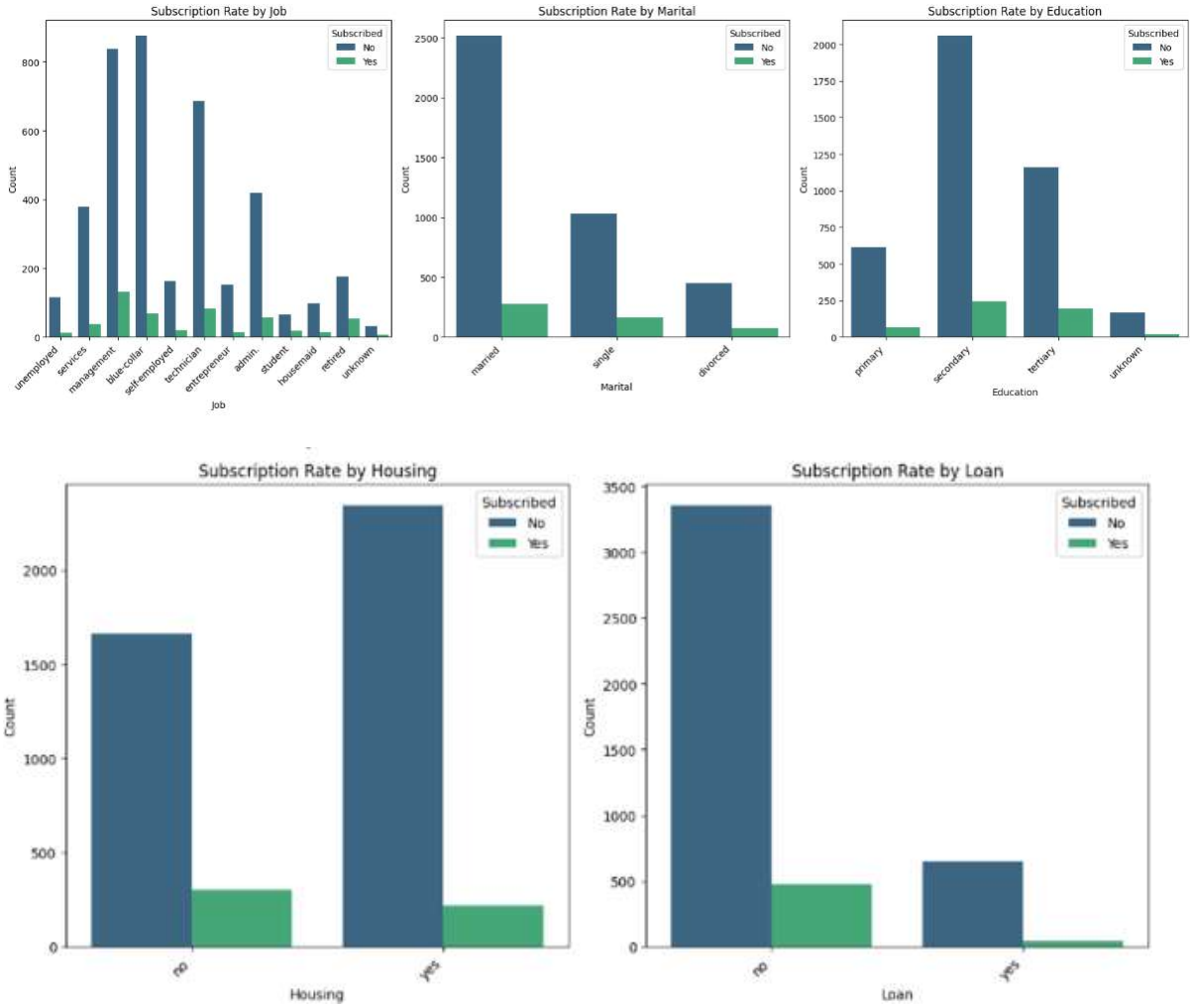


Figure 3.2 The categorical relationship between demographics and subscriptions levels.

2.2.5 Numerical Feature Analysis and Correlations

Correlation analysis shows that call duration is the strongest predictor of subscription (0.40), with longer calls associated with higher likelihood of subscription. Pdays and previous show moderate correlations (0.58), indicating that clients who were contacted recently or multiple times are more likely to subscribe. In contrast, features such as age, balance, day, and campaign show weak linear relationships, suggesting that non-linear patterns dominate. As a result, tree-based models are recommended to capture these complex relationships, while logistic regression can provide interpretable insights into feature effects.

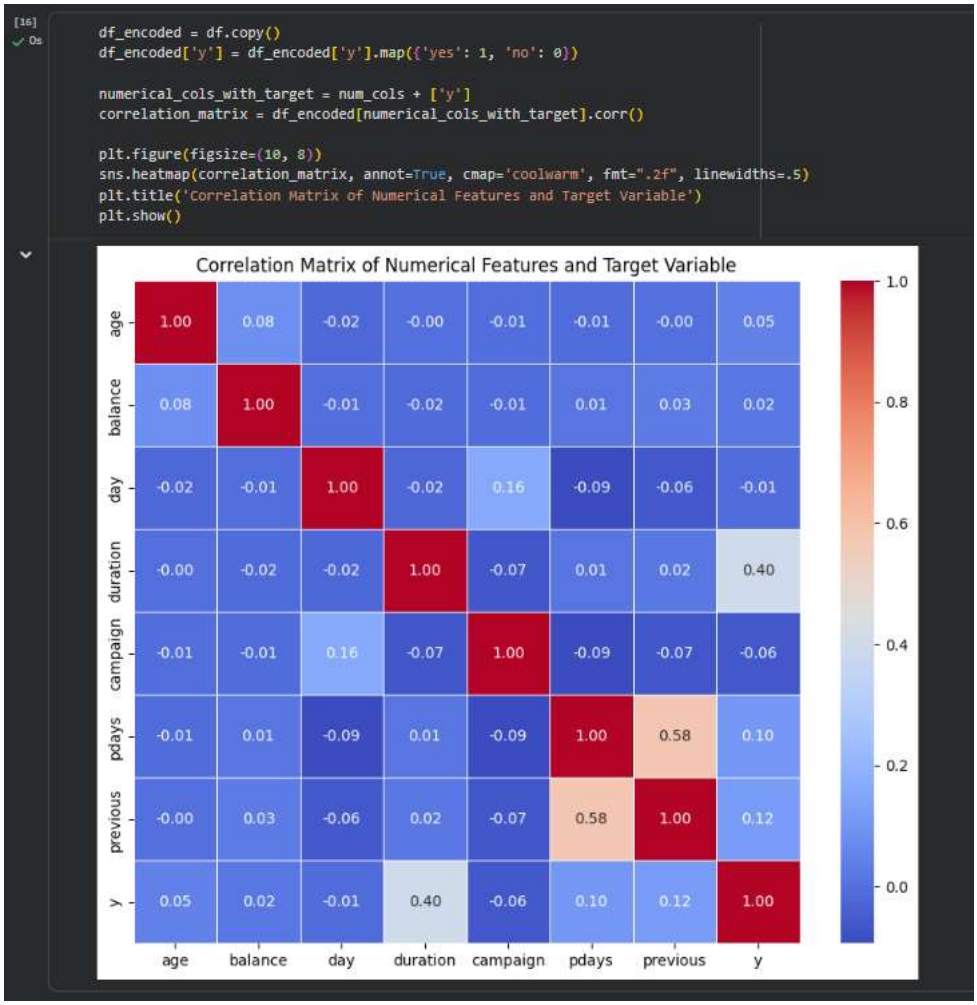


Figure 4 Correlation matrix of numerical features

2.2.6 Call Duration Analysis

Subscribers consistently have longer call durations (1000+ seconds), with a higher median, wider spread and more high-end outliers. Short calls almost never correspond to a successful subscription, highlighting duration as a one of the strongest predictors in the dataset. Due to strong skewness, models must treat this variable non-linearly or through transformation to prevent bias.

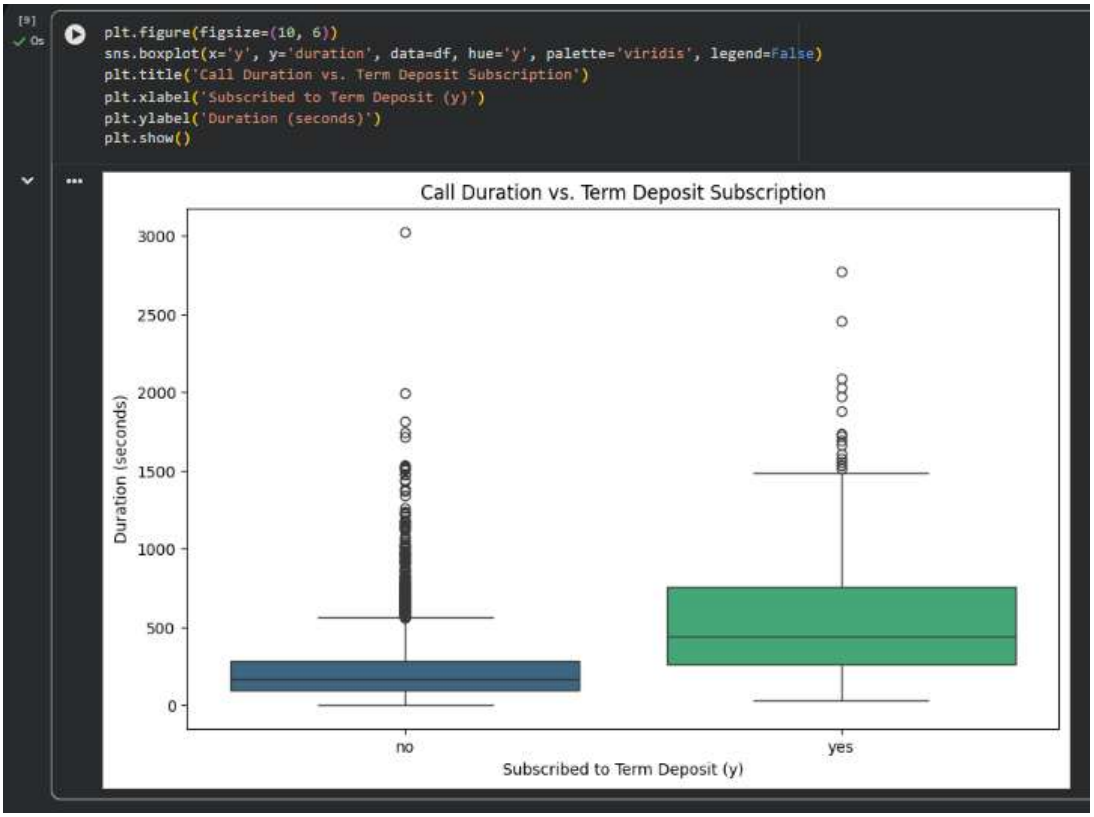


Figure 5 Duration call Impact

2.3 EDA Conclusions

The exploratory data analysis of the bank dataset reveals several key insights into term deposit subscriptions:

1. **Imbalanced Target Variable:** 88.48% clients do not subscribe. that must be considered in predictive modelling.
2. **Demographics Influence:** Older, tertiary-educated, retired and single clients are more likely to subscribe.
3. **Financial Stability Matters:** Clients with higher account balances and no existing loans are more likely to subscribe.
4. **Call Duration as a Behavioural Indicator:** Longer calls are the strongest predictor of subscription.
5. **Numerical Features:** Aside from call duration, numerical variables show weak linear correlations with subscriptions, suggesting that non-linear relationships,
6. **Modelling Implications:** Address class imbalance, consider non-linear relationships, and integrate both numerical and categorical features.

3 Experimental Design & Preprocessing

3.1 Algorithms

The algorithms we used Logistic Regression, 'a supervised machine learning algorithm in data science. It is a type of classification algorithm that predicts a discrete or categorical outcome.' (Lee, 2025).[1] and Random Forest, 'an ensemble learning method that constructs multiple decision trees and combines their outputs to make predictions.' (Team, 2025)[2] , because they are well-suited for analysing binary classification problems. In this dataset, the aim is to predict whether a customer will subscribe to a term deposit (y = yes or no). Logistic Regression was chosen for its strong ability to model binary outcomes, while Random Forest was chosen for its ability to capture non-linear relationships, and reducing overfitting through multiple decision trees. The use of both allows us to compare interpretability with predictive performance.

3.2 Evaluation Metrics

To evaluate the performance of our models, we will use Accuracy, Precision, Recall and F1-Score. Accuracy measures the overall percentage of correct predictions, but on its own it can be misleading for this dataset because it has class imbalance, meaning there are many more "no" responses than "yes". Because of this, we also use Precision and Recall. Precision measures how many of the customers predicted as "yes" actually subscribed, which is important because incorrectly targeting customers can waste marketing resources. Recall shows how well the model identifies customers who subscribed, which helps ensure potential clients are not missed. F1-Score combines both Precision and Recall into one balanced metric, giving a more reliable evaluation when the dataset is imbalanced. Using all four metrics allows us to judge the models fairly and understand their strengths beyond just accuracy.

3.3 Preprocessing Steps

Preprocessing is, 'processing performed on raw data to prepare it for another data processing procedure. so, it can be used effectively by our models.' (Lawton, 2022).[3] This includes, cleaning, handling missing values, encoding categorical variables, scaling numerical variables, removing outliers and fixing any class imbalance. The aim of preprocessing is to make the dataset structured for accurate model training.

---

[1] Lee, F. (2025). *Logistic Regression*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/logistic-regression.

[2] Team, S. (2025). *Random Forest Algorithm in Machine Learning With Example - SitePoint*. [online] Sitepoint.com. Available at: https://www.sitepoint.com/random-forest-algorithm-in-machine-learning/.

[3] Lawton, G. (2022). *Data Preprocessing: Definition, Key Steps and Concepts*. [online] SearchDataManagement. Available at: https://www.techtarget.com/searchdatamanagement/definition/data-preprocessing.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score
from sklearn.metrics import RocCurveDisplay

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

from imblearn.over_sampling import SMOTE
import matplotlib.pyplot as plt
```

This section imports all necessary libraries for data preparation, model building and performance evaluation, including tools for loading and processing data, splitting and scaling it, handling class imbalance with SMOTE, training models such as Logistic Regression, Decision Tree and SVC, and visualising results with matplotlib.

```
df = pd.read_csv('/content/bank.csv', sep=';', quotechar='"')
df.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	30	unemployed	married	primary	no	1787	no	no	cellular	19	oct	79	1	-1	0	unknown	no
1	33	services	married	secondary	no	4789	yes	yes	cellular	11	may	220	1	339	4	failure	no
2	35	management	single	tertiary	no	1350	yes	no	cellular	16	apr	185	1	330	1	failure	no
3	30	management	married	tertiary	no	1476	yes	yes	unknown	3	jun	199	4	-1	0	unknown	no
4	59	blue-collar	married	secondary	no	0	yes	no	unknown	5	may	226	1	-1	0	unknown	no

This code loads the Bank Marketing dataset using the correct separator (;) and quote character (“). The df.head function prints the first five rows, giving a preview of the dataset.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4521 entries, 0 to 4520
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        4521 non-null   int64
 1   job        4521 non-null   object
 2   marital    4521 non-null   object
 3   education  4521 non-null   object
 4   default    4521 non-null   object
 5   balance    4521 non-null   int64
 6   housing    4521 non-null   object
 7   loan       4521 non-null   object
 8   contact    4521 non-null   object
 9   day        4521 non-null   int64
 10  month      4521 non-null   object
 11  duration   4521 non-null   int64
 12  campaign   4521 non-null   int64
 13  pdays      4521 non-null   int64
 14  previous   4521 non-null   int64
 15  poutcome   4521 non-null   object
 16  y          4521 non-null   object
dtypes: int64(7), object(10)
memory usage: 600.6+ KB
```

The df.info command gives a summary of the dataset, including the number of rows, columns, datatypes and whether there are any missing values.

```
df.columns
```

```
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
       'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'y'],
      dtype='object')
```

The df.columns line displays all the column names in the dataset. It helps confirm the dataset has loaded correctly and shows which features are available for preprocessing and modelling.

Data Cleaning:

```
cols = df.columns
new_cols = []

for column in cols:
    proper_col = column.title()
    clean_col = proper_col.replace("-", "_")
    new_cols.append(clean_col)

df.columns = new_cols

df.head()
```

	Age	Job	Marital	Education	Default	Balance	Housing	Loan	Contact	Day	Month	Duration	Campaign	Pdays	Previous	Poutcome	Y
0	30	unemployed	married	primary	no	1787	no	no	cellular	19	oct	79	1	-1	0	unknown	no
1	33	services	married	secondary	no	4789	yes	yes	cellular	11	may	220	1	339	4	failure	no
2	35	management	single	tertiary	no	1350	yes	no	cellular	16	apr	185	1	330	1	failure	no
3	30	management	married	tertiary	no	1476	yes	yes	unknown	3	jun	199	4	-1	0	unknown	no
4	59	blue-collar	married	secondary	no	0	yes	no	unknown	5	may	226	1	-1	0	unknown	no

This block of code loops through each column name and formats it by capitalising the first letter using (title()) and replacing any hyphens with underscores. The updated column names are then assigned back to the dataframe.

Data Types:

```
df.dtypes
```

	0
Age	int64
Job	object
Marital	object
Education	object
Default	object
Balance	int64
Housing	object
Loan	object
Contact	object
Day	int64
Month	object
Duration	int64
Campaign	int64
Pdays	int64
Previous	int64
Poutcome	object
Y	object

dtype: object

The function df.types shows the data type of each column.

Missing Data:

```
df_cleaned = df.dropna()
```

Even though the dataset has no missing values, the df_cleaned line creates a cleaned version of the dataframe by removing any rows that may contain null values.

```
df.isnull().sum()
```

	0
Age	0
Job	0
Marital	0
Education	0
Default	0
Balance	0
Housing	0
Loan	0
Contact	0
Day	0
Month	0
Duration	0
Campaign	0
Pdays	0
Previous	0
Poutcome	0
Y	0

dtype: int64

This code checks every column in the dataset for missing values.

```
missing_pct = round(df.isnull().sum()/len(df)* 100,1)
print(missing_pct)
```

```
Age          0.0
Job          0.0
Marital      0.0
Education    0.0
Default      0.0
Balance      0.0
Housing      0.0
Loan         0.0
Contact      0.0
Day          0.0
Month        0.0
Duration     0.0
Campaign     0.0
Pdays        0.0
Previous     0.0
Poutcome     0.0
Y            0.0
dtype: float64
```

This code calculates the percentage of missing values in each column.

Skewness:

```
numeric_cols = df.select_dtypes(include=['int64', 'float64'])
numeric_cols.skew()
```

	0
Age	0.699501
Balance	6.596431
Day	0.094627
Duration	2.772420
Campaign	4.743914
Pdays	2.717071
Previous	5.875259

dtype: float64

This section of code selects all numerical columns in the dataset and calculates their skewness.

```
print("Mean:", df['Age'].mean())
print("Median:", df['Age'].median())
print("Mode:", df['Age'].mode()[0])
```

```
Mean: 41.17009511170095
Median: 39.0
Mode: 34
```

This code calculates the mean, median and mode of the Age column.

```
print("Skewness:", df['Age'].skew)
print("Outliers:", df['Age'].quantile(0.99))
```

```
Skewness: <bound method Series.skew of 0       30
1       33
2       35
3       30
4       59
        ..
4516    33
4517    57
4518    57
4519    28
4520    44
Name: Age, Length: 4521, dtype: int64>
Outliers: 72.0
```

This code calculates the skewness of the Age column and identifies potential outliers using the 99th percentile. Skewness, 'is a key statistical measure that shows how data is spread out in a dataset' (GeeksforGeeks, 2023)[4], and the output suggests a slightly positive skew, meaning some customers are older than the typical range.

```
median_units = df['Age'].median()
df['Age']=df['Age'].fillna(median_units)
```

This code calculates the median Age and uses it to fill any missing Age values.

```
df['Age'].isnull().sum()
```

```
np.int64(0)
```

This code checks how many missing values exist in the Age column.

Data Scaling:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
```

The function pandas is used to handle and manipulate the dataset, while StandardScaler is needed for scaling numerical features so that they are on a similar range before training the model.

```
bank_df = pd.read_csv("bank.csv", sep=';')
```

This line loads the Bank Marketing dataset into a dataframe called bank_df.

[4] GeeksforGeeks (2023). *Skewness Measures and Interpretation*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-science/skewness-measures-and-interpretation/.

```
num_cols = ['age', 'balance', 'duration', 'campaign', 'pdays', 'previous']

scaler = StandardScaler()
bank_df[num_cols] = scaler.fit_transform(bank_df[num_cols])

bank_df.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	-1.056270	unemployed	married	primary	no	0.121072	no	no	cellular	19	oct	-0.711861	-0.576829	-0.407218	-0.320413	unknown	no
1	-0.772583	services	married	secondary	no	1.118644	yes	yes	cellular	11	may	-0.169194	-0.576829	2.989044	2.041734	failure	no
2	-0.583458	management	single	tertiary	no	-0.024144	yes	no	cellular	16	apr	-0.303898	-0.576829	2.899143	0.270124	failure	no
3	-1.056270	management	married	tertiary	no	0.017726	yes	yes	unknown	3	jun	-0.250017	0.387967	-0.407218	-0.320413	unknown	no
4	1.686036	blue-collar	married	secondary	no	-0.472753	yes	no	unknown	5	may	-0.146102	-0.576829	-0.407218	-0.320413	unknown	no

This code scales the numerical columns, so they have a mean of 0 and a standard deviation of 1.

One Hot Encoding:

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
```

This code imports tools needed for preprocessing, using OneHotEncoder to convert categorical data, and StandardScaler to scale numerical features. ColumnTransformer allows us to apply different preprocessing steps to each type of column simultaneously.

```
categorical_columns = ['Job', 'Marital', 'Education', 'Default', 'Housing', 'Loan', 'Contact', 'Month', 'Poutcome']

numeric_columns = ['Age','Balance','Day','Duration','Campaign','Pdays','Previous']
```

This section separates categorical and numerical columns in the dataset.

```
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(drop='first'), categorical_columns),
        ('num', StandardScaler(), numeric_columns)
    ]
)
```

This code creates a process that automatically applies one-hot encoding to the categorical columns and standard scaling to the numerical columns.

3.4 Limitations:

Bank marketing dataset comes with several limitations that can affect the performance and reliability of the models. One of the main limitations is class imbalance, as most of the customers in the dataset did not subscribe to a term deposit. This uneven distribution makes it harder for the model to learn patterns associated with the minority class, it also increases the model predicting “no” because it is more common. Another limitation involves noisy attributes. Several features such as, Balance, Campaign and Previous, contain highly skewed values or irregular patterns that do not follow a clear structure. These attributes can reduce the model's ability to detect meaningful relationships. Finally, the dataset size is relatively small considering how complex customer behaviour is. With just over 4000 rows there is limited variation for the model to learn from, this may affect how well it generalises to new customers outside the data sample.

4 Model Development

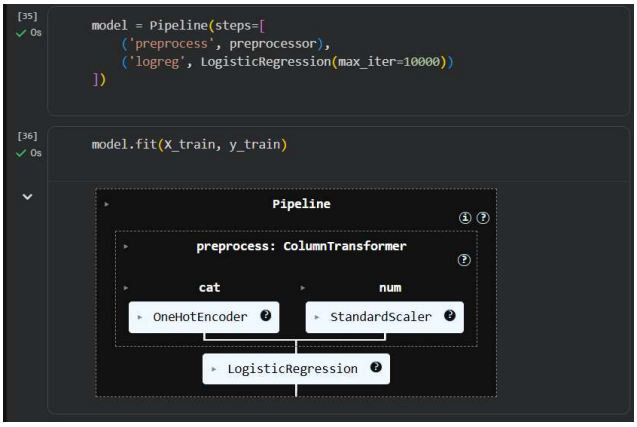
4.1 Data Splitting

```
[21]
✓
    X = df.drop("Y", axis=1)
    y = df["Y"]

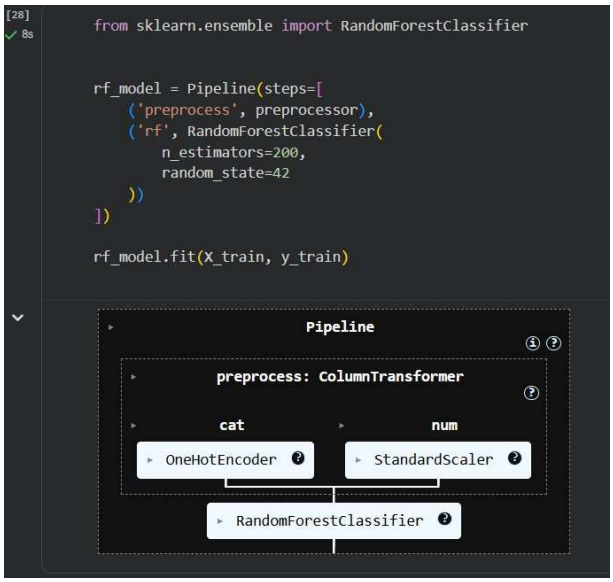
[22]
✓
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

4.2 Model Training

4.2.1 Logistic Regression

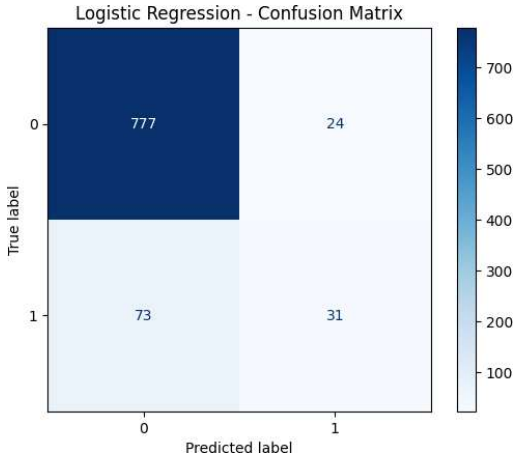


4.2.2 Random Forest



4.3 Initial Results

4.3.1 Logistic Regression Results

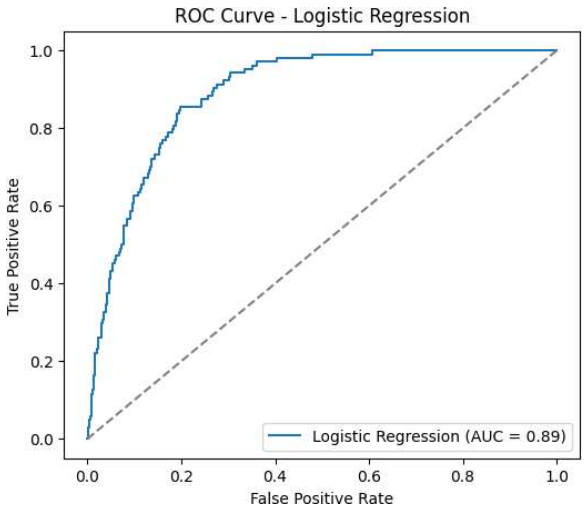


The Logistic Regression model shows that it correctly classifies most “no” cases with 777 instances and misclassifies only 24. It also shows that 31 out of 104 positive cases were predicted accurately, showing the model performs poorly with the “yes” cases as it wrongly identifying 73 customers not subscribing to a term deposit when they did.

Logistic Regression - Classification Report:

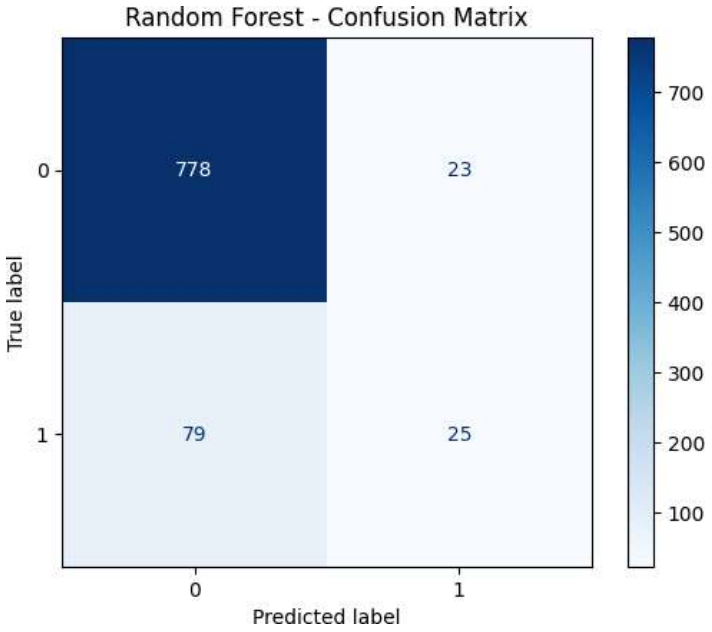
	precision	recall	f1-score	support
no	0.91	0.97	0.94	801
yes	0.56	0.30	0.39	104
accuracy			0.89	905
macro avg	0.74	0.63	0.67	905
weighted avg	0.87	0.89	0.88	905

The Classification Report reflects a similar output, the “no” class achieves high precision as when they model assumes no, it is usually correct. However, the precision for the “yes” class it significantly lower at 0.56 and a recall of 0.30. This dramatic decrease in correctly identifying positive cases rather than negative cases can be due to the class imbalance of there being a lot more “no” cases that were used to train the model.



To further evaluate the model, the ROC curve and AUC score was implemented. The curve shows how well the model distinguishes between subscribers and non-subscribers; the ROC curve rises high above the baseline indicating that the model performs significantly better compared to random guessing. The model achieved an AUC score of 0.8918419283587822, showing strong separability between the two outcomes.

4.3.2 Random Forest Results

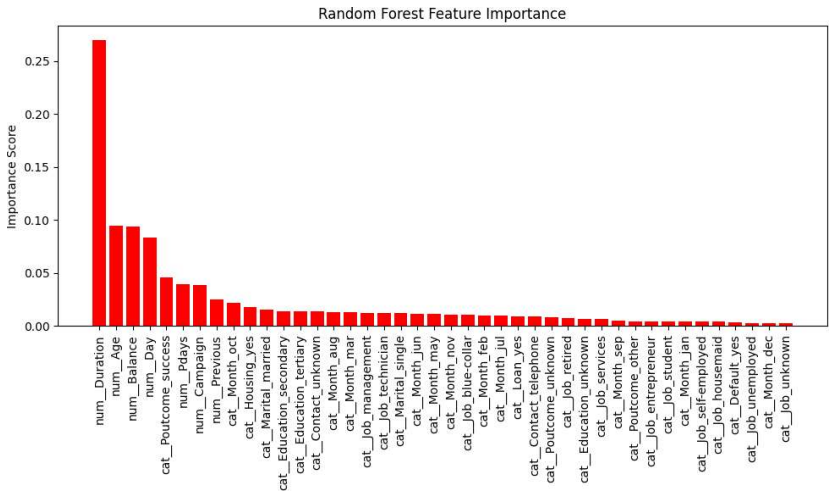


Similar to the Logistic Regression model, the Random Forest model has a strong performance on the majority “no” class, predicting correctly 778 classes whilst only misclassifying 23. However, the Confusion Matrix shows a greater difficulty in identifying “yes” cases only correctly identifying 25 out of 104 positive instances, this indicates low recall as the model misses many people who subscribed (“yes”).

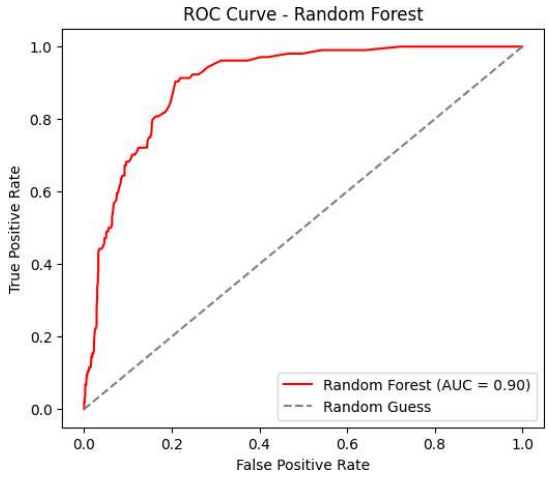
Random Forest - Classification Report:

	precision	recall	f1-score	support
no	0.91	0.97	0.94	801
yes	0.52	0.24	0.33	104
accuracy			0.89	905
macro avg	0.71	0.61	0.63	905
weighted avg	0.86	0.89	0.87	905

This is further supported with the Classification Report as it details the “yes” class has a recall of a very low score of 0.24. It performed significantly well in the “no” class with a 0.91 in precision, a 0.97 in recall and 0.94 in f1-score, again very close to Logistic Regression.



A feature importance plot was generated from the Random Forest model to reveal which variables had the greatest influence on predicting whether a customer subscribed. The most dominant feature is num_Duration which indicates that the length of the last contact call is the strongest predictor whether a customer subscribes. The next most influential features are num_Age and num_Balance, however beyond these attributes the importance scores drop sharply. This decline highlights that the model relies heavily on small set of variables while most features add little value. Adding this plot was important as it provides interpretability on why the model performs well, confirming it was because of a handful of features that carry most predictive power.



The Random Forest classifier was also assessed further using the ROC curve and AUC score. Unlike the linear decision of Logistic Regression, Random Forest uses multiple decision trees which allows it to capture more complex patterns in the data. The ROC curve rises toward the upper-left corner indicating the model's strong performance to separate subscribers and non-subscribers across varying probability thresholds. The model also acquired an AUC score of 0.9025076827043119, showing impressive power in separating the “yes” and “no” outcomes; confirming that the ensemble approach of multiple decision trees provides a more robust performance compared to the simpler methods.

5 Model Tuning

5.1 Identifying Model Weaknesses

The Logistic Regression and Random Forest models both exhibit weaknesses that are seen in the results. A major limitation of LR is that it assumes that the linearity between the dependent and independent variable (GeeksforGeeks, 2020)[5]. This means that the model expects each feature to influence the prediction in a straight line. However, many of the relationships between attributes are nonlinear and interact with each to a certain extent, thus making it difficult to obtain complex relationships. Additionally Logistic Regression is sensitive toward multicollinearity between independent variables (GeeksforGeeks, 2020)[6]. Multicollinearity occurs when two or more variables in the model are highly correlated with each other, as a result the model struggles to separate individual effects, making the model less reliable and harder to interpret.

Random Forests main limitation is that they are much more complex compared to Logistic Regression, they are relatively opaque and difficult to interpret when there is a forest of decision trees generated (IBM, 2021)[7]. Without hyperparameters controlling tree depth and estimators, the model can overfit the training data, making it unreliable. Which can be seen in the Classification Report with a very high performance on the majority class and a very low performance on the minority class.

5.2 Model Improvement (Hyperparameter Tuning)

5.2.1 Logistic Regression tuning

```
[30]
✓ 5s
from sklearn.model_selection import GridSearchCV

logreg_param_grid = {
    "logreg__C": [0.001, 0.01, 0.1, 1, 10]
}

logreg_grid = GridSearchCV(
    estimator=model,
    param_grid=logreg_param_grid,
    cv=5,
    scoring="f1_macro"
)

logreg_grid.fit(X_train, y_train)

print("Best Logistic Regression Parameters:", logreg_grid.best_params_)

Best Logistic Regression Parameters: {'logreg__C': 10}
```

This code improves the Logistic Regression model through hyperparameter tuning. The logreg__C refers to the C parameter in the pipeline. The smaller the value (0.001) the stronger regularisation and the larger it

[5] GeeksforGeeks (2020). *Advantages and Disadvantages of Logistic Regression*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-science/advantages-and-disadvantages-of-logistic-regression/.

[6] GeeksforGeeks (2020). *Advantages and Disadvantages of Logistic Regression*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-science/advantages-and-disadvantages-of-logistic-regression/.

[7] IBM (2021). *What is random forest?* [online] Ibm.com. Available at: https://www.ibm.com/think/topics/random-forest.

is the weaker the regularisation. The dictionary tells the GridSearchCV to try the model using these 5 different C values and finds which performs best, as an extremely large C can overfit and an extremely small C can underfit.

The next piece of code states the model being tuned, which is the pipeline that ends with a Logistic Regression estimator. Cv = 5 uses 5 – fold cross – validation, this means the training data is split into 5 different folds and the model trained 5 times, using 4 folds for training and 1 fold for validation (Brownlee, 2023)[8]. The scoring = "f1_macro" tells the GridSearchCV to choose the best hyperparameter based on the macro F1 score which uses the "yes" and "no" classes equally.

The grid search is then run training each C value 5 times and the C value with the highest macro F1 score is printed which is 10.

5.2.2 Random Forest Tuning



The first part of the tuning defines which hyperparameter values GridSearchCV is to test. The n_estimators tests the model with 100 and 150 trees, the more trees there are the more stable but slower the model is. Max_depth tests how deep each decision tree can go, None is that trees can grow as deep as possible which may lead to overfitting and 10, which limits the depth to 10. Lastly, the min_samples_split is the maximum number of samples requires to split the node, which is either 2 or 5, the higher the number the fewer the splits which reduces over fitting but makes the model less complex.

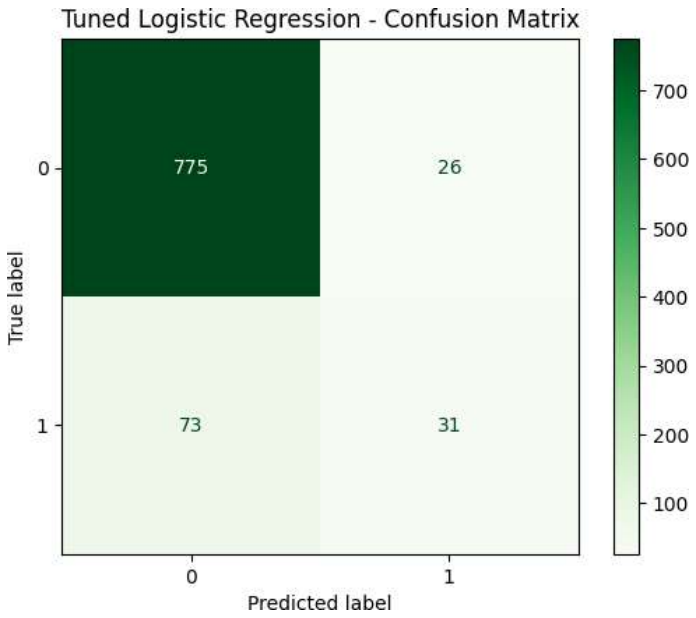
The rest of the code is very similar to the Logistic Regression tuning with the 5 – fold cross – validation and the best model will be chosen with the macro F1 score.

The code is then run and the highest performing hyperparameter combination is produced; the max_depth is None, min_samples_split is 2 and finally the n_estimators to be 100.

5.3 Improved Results

5.3.1 Logistic Regression – After Tuning Results

[8] Brownlee, J. (2023). *A Gentle Introduction to k-fold Cross-Validation*. [online] Machine Learning Mastery. Available at: https://machinelearningmastery.com/k-fold-cross-validation/.



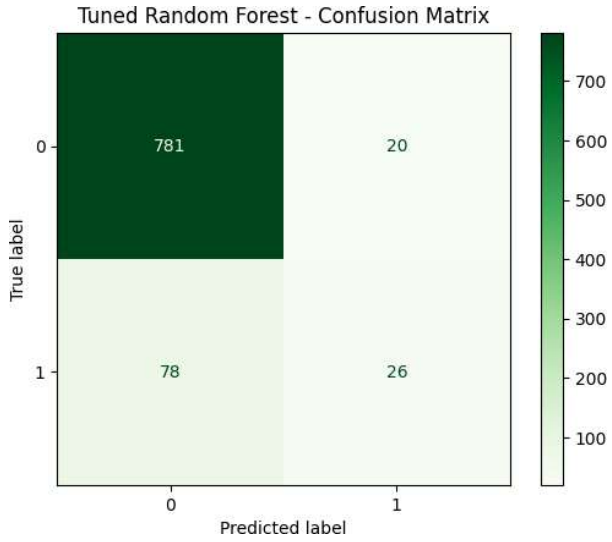
After hyperparameter tuning, the model did not show any improvements, and the Confusion Matrix remains nearly unchanged except for the predicted and true label of the "no" category to decrease by 2 instances. This highlights the main limitation of Logistic Regression which is the models' inherent assumptions of linearity between attributes and adjusting the regularisation strength cannot overcome it.

Tuned Logistic Regression - Classification Report:

	precision	recall	f1-score	support
no	0.91	0.97	0.94	801
yes	0.54	0.30	0.39	104
accuracy			0.89	905
macro avg	0.73	0.63	0.66	905
weighted avg	0.87	0.89	0.88	905

The Classification Report further details that even though the tuned model still performs well on the majority class, it struggles to correctly identify the minority class. Again, demonstrating that hyperparameter tuning is insufficient by itself to address the structural weaknesses of Logistic Regression.

5.3.2 Random Forest – After Tuning Results



The Random Forest tuning did slightly improve the results of the Confusion Matrix. 7 instances more were correctly identified as “no” and 1 more was identified as “yes”. Even though the optimal parameters were selected by the GridSearchCV, like the Logistic Regression model, it still did not substantially alter the model’s behaviour.

Tuned Random Forest - Classification Report:

	precision	recall	f1-score	support
no	0.91	0.98	0.94	801
yes	0.57	0.25	0.35	104
accuracy			0.89	905
macro avg	0.74	0.61	0.64	905
weighted avg	0.87	0.89	0.87	905

The overall macro F1-score increased only marginally from 0.63 to 0.64 and the “yes” class recall increased from 0.24 to 0.25. This shows that the model continued to perform very well on the majority class as it achieved a higher recall of 0.98 from 0.97 but still misclassifies the minority class. This suggests that the underlying class imbalance can not be resolved with hyperparameter tuning alone, just like the Logistic Regression model; and that additional techniques will be needed to improve the minority class recall.

6 Conclusion and Reflective Commentary

6.1 overall Conclusion

The key objective of this project was to determine whether supervised machine-learning techniques could predict the probability of a customer subscribing to a bank term deposit using demographic, financial, and campaign-interaction features. Across the modelling process, Logistic Regression and Random Forest worked as appropriate foundations for comparison, reflecting two contrasting priorities: interpretability and predictive complexity.

The outcomes demonstrated that while both models performed reliably on the majority class ("no"), their ability to detect true positive subscriptions was significantly lower. This result confirms a key observation from the exploratory analysis: the dataset was heavily imbalanced, with considerably more non-subscribers than subscribers.

Although model behaviour gradually improved by hyperparameter tuning, especially within the Random Forest classifier, these improvements were only slight, indicating that class imbalance and feature noise posed significant limitations on model learning.

6.2 Reflection on Learning and Methodology

Developing the models needed to shift past simple algorithm choice and instead developing a deeper understanding of the dataset itself how it was structured, how preprocessing decisions shaped the inputs, and how each choice introduced trade oofs that affected the final performance. One of the most useful conclusions gained from this process was recognizing the extent to which poor data quality may overpower even sophisticated modelling attempts. As an example, imbalances in the target variable and distorted numerical distributions affected both the model evaluation and the reliability of performance metrics. These variables demonstrate that machine learning is not just about training a model but about creating the right conditions for the model to learn meaningful patterns rather than noise. As a result, a significant part of the development process focused not on adjusting model parameters, but on preparing the dataset in a way that gave the algorithms a fair opportunity to expand and perform effectively.

The evaluation metrics themselves were a second important lesson. Accuracy alone proved insufficient, and the exercise reinforced why metrics such as memory, accuracy, and F1- score are essential for imbalanced classification tasks. Understanding these measures helped clarify model results beyond numeric outputs by forcing consideration of real-world consequences: missed predications represent potential lost customers, and false positives reflect inefficient campaign spending.

6.3 Challenges, Limitations and Ethical Considerations

Multiple difficulties arose during the project. The class imbalance restricted the accuracy of positive predictions, despite the implementation of tuning and weighting modifications. As second challenge arose from nonlinear associations between features, which Logistic Regression could not adequately capture. In the meantime, while Random Forest managed complexity more effectively, understanding its workings became harder a common trade-off when weighing clarity against predictive performance.

Ethical factors also held significant importance. Due to the presence of demographic and socioeconomic indicators in the dataset, deploying the model may potentially reinforce inequalities or unjustly target

individuals at risk. Guaranteeing equity, avoiding biased results, and upholding clarity in automated decision-making are essential factors for any practical implementation.

6.4 Future Work

Future developments of this project might concentrate on enhancing model fairn3ess, recall effectiveness, and feature representation. Potential pathways for development encompass:

1. Utilizing class-balancing techniques that are cost-sensitive.
2. Testing gradient boosting models (XGBoost, LightGBM), which often exceed baseline ensembles in performance on tabular structured data.
3. Implementing feature selection and/or elaborate feature engineering to minimize noise and enhance signal significance.

Assessing explainability framework like SHAP to find a balance between complexity and interpretability.

6.5 Reflection on individual learning

Engaging in this project enhanced my comprehension of how data dynamics affect modelling results. A crucial lesson was that successful machine-learning pipelines rely not just on algorithms but also on diligent preparation, cleaning, encoding, scaling, deciphering patterns, and avoiding the inclination to believe the model will fix data flaws.

Another aspect of personal growth is understanding when a model's limitations arise from data constraints instead of a coding mistake. The persistent recall issue highlighted that not every dataset is balanced, complete, or ideally arranged and that an accurate evaluation recognizes limitations instead of hiding them.

Ultimately, the project-built confidence in autonomously manoeuvring through the extensive modelling process from initial exploration to validating assumptions, refining model versions, and assessing progress with relevant evidence rather than relying solely on intuition. This experience has enhanced both technical skills and critical thinking related to machine learning design and assessment.

7 Bibliography

Archive.ics.uci.edu. (n.d.). *UCI Machine Learning Repository*. [online] Available at: https://archive.ics.uci.edu/dataset/222/bank+marketing.

Lee, F. (2025). *Logistic Regression*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/logistic-regression.

Team, S. (2025). *Random Forest Algorithm in Machine Learning With Example - SitePoint*. [online] Sitepoint.com. Available at: https://www.sitepoint.com/random-forest-algorithm-in-machine-learning/.

Lawton, G. (2022). *Data Preprocessing: Definition, Key Steps and Concepts*. [online] SearchDataManagement. Available at: https://www.techtarget.com/searchdatamanagement/definition/data-preprocessing.

GeeksforGeeks (2023). *Skewness Measures and Interpretation*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-science/skewness-measures-and-interpretation/.

Galarnyk, M. (2022). *Train Test Split: What it Means and How to Use It | Built In*. [online] builtin.com. Available at: https://builtin.com/data-science/train-test-split.

IBM (2024a). *Data leakage machine learning*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/data-leakage-machine-learning.

scikit-learn.org. (n.d.). *sklearn.pipeline.Pipeline — scikit-learn 0.24.1 documentation*. [online] Available at: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html.

scikit-learn (2014). *Logistic Regression*. [online] Scikit-learn.org. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

GeeksforGeeks (2024). *What is Scikitlearn Random State in Splitting Dataset?* [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/what-is-scikit-learn-random-state-in-splitting-dataset/#what-israndom_state.

GeeksforGeeks (2020). *Advantages and Disadvantages of Logistic Regression*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-science/advantages-and-disadvantages-of-logistic-regression/.

IBM (2021). *What is random forest?* [online] Ibm.com. Available at: https://www.ibm.com/think/topics/random-forest.

Brownlee, J. (2023). *A Gentle Introduction to k-fold Cross-Validation*. [online] Machine Learning Mastery. Available at: https://machinelearningmastery.com/k-fold-cross-validation/.

Meissner, T., Gassmann, X., Faure, C. and Schleich, J. (2022) '*Individual characteristics associated with risk and time preferences: A multi-country representative survey*', Journal of Risk and Uncertainty. Available at: Individual characteristics associated with risk and time preferences: A multi country representative survey | Journal of Risk and Uncertainty

Silinskas, G., Ranta, M. and Wilska, T.-A. (2021) *'Financial Behaviour Under Economic Strain in Different Age Groups: Predictors and Change Across 20 Years'*, Journal of Consumer Policy, 44(2), pp. 235–257.
Available at: Financial Behaviour Under Economic Strain in Different Age Groups: Predictors and Change Across 20 Years | Journal of Consumer Policy

Zeng, Y., Chen, C., Wang, C. and Zou, J. (2025) *'Education investment, social security, and household financial market participation'*, Finance Research Letters, 77, p. 107124.

Available at: Education investment, social security, and household financial market participation - ScienceDirect